

SIMATS ENGINEERING

ASSESSMENT TOOL 1

DSA0202 - COMPUTER VISION WITH OPENCV

Name : NAVADEEP R

Reg No : 192424110

Faculty Incharge : Mayuranathan M

CO2 – ASSESSMENT TOOL-1

Analytical Problem Solving

Code of Conduct:

I Navadeep R (192424110) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Navadeep R

Question 1: Salt-and-Pepper Noise Removal Before Edge Detection

Scenario: A grayscale image of size 1024×1024 is captured using a surveillance

camera under poor lighting conditions. The image is corrupted by salt-and-pepper

noise with a noise density of 0.05. This means that some pixels are randomly changed

to either black or white intensity values. The image has to be preprocessed before

applying an edge detection algorithm. Since edge detection is sensitive to abrupt

intensity variations, the selected filter should remove noise effectively while preserving

important edge details.

Answer the following:

a. Identify the most suitable filtering technique for removing salt-and-pepper noise

without causing significant edge blurring.

b. Explain why the selected filter performs better than an averaging filter for impulse

noise removal.

c. If a 3×3 median filter is applied, estimate the probability that a noisy center pixel

will be corrected. Clearly state your assumptions.

d. Estimate the total number of noisy pixels present in the original image.

- e. Estimate the expected number of noisy pixels remaining after median filtering.
- f. Explain how the preprocessing strategy would change if the same image was affected by Gaussian noise instead of salt-and-pepper noise.

Scenario

A grayscale image of size 1024×1024 is corrupted with salt-and-pepper noise having a noise density of **0.05 (5%)**. The image must be preprocessed before edge detection.

a. Most suitable filtering technique

The most suitable filter is the **Median Filter**, preferably a 3×3 **median filter**.

A median filter replaces each pixel with the median value of the pixels in its neighborhood.

Reason: It is highly effective against salt-and-pepper noise while preserving sharp edges better than linear smoothing filters.

b. Why median filtering is better than averaging filtering

An **averaging filter** calculates the mean of neighboring pixels. Therefore, extreme black or white noise values can influence the average and produce blurred edges.

A **median filter** sorts the neighborhood pixel values and selects the middle value.

For example:

20, 21, 22, 23, 255, 24, 22, 21, 20

The median is **22**, so the extreme noisy value **255** is removed.

Therefore:

- Median filter → removes impulse noise effectively.
- Averaging filter → reduces noise but tends to blur edges.
- Median filter → better preserves important edge details.

c. Probability that a noisy center pixel will be corrected

Assume:

- Noise density, $p = 0.05$
- Probability of a clean pixel = 0.95
- A 3×3 filter has 9 pixels.
- The center pixel is known to be noisy.
- The noisy center pixel is corrected when the other **8 neighboring pixels contain at least 5 clean pixels**.

Probability:

$$P(X \geq 5) = \sum_{k=5}^8 \binom{8}{k} (0.95)^k (0.05)^{8-k}$$

This gives approximately:

$$P \approx 0.99998$$

Therefore, the estimated probability of correction is approximately:

$$\boxed{99.998\%}$$

Assumption: The noise is independently and randomly distributed, and the median is assumed to correctly recover the center pixel when the majority of its neighborhood is clean.

d. Total number of noisy pixels

Image size:

$$1024 \times 1024 = 1,048,576$$

Noise density:

$$0.05$$

Therefore:

$$1,048,576 \times 0.05 = 52,428.8$$

Hence, approximately:

$$\boxed{52,429 \text{ noisy pixels}}$$

e. Expected noisy pixels remaining

Using the probability that a noisy center pixel is **not corrected**:

$$1 - 0.99998 \approx 0.00002$$

Expected remaining noisy pixels:

$$52,429 \times 0.00002 \approx 1$$

Therefore, approximately:

1 noisy pixel

Note: This is an idealized estimate. Actual results depend on the distribution of noise, image content, and border effects.

f. If the image contains Gaussian noise

For Gaussian noise, a **Gaussian filter** is generally more appropriate than a median filter.

The Gaussian filter performs a weighted averaging operation, where nearby pixels receive higher weights.

Typical pipeline:

Noisy Image → Gaussian Filter → Edge Detector

For salt-and-pepper noise:

Noisy Image → Median Filter → Edge Detector

Thus, the filter should be selected according to the noise model.

Question 2: Discretization of a Continuous Image

Scenario : An analog grayscale image has a smooth sinusoidal intensity variation along the horizontal direction. The intensity changes continuously with respect to the spatial coordinate x , while it remains constant along the y -direction. The image is defined over a spatial range of 0 to 255 in both directions. This continuous image has to be converted into a digital image for storage and further processing in a computer vision system.

Answer the following:

- a. Explain how the continuous image can be converted into a digital image using sampling and quantization.
- b. Identify the spatial variation pattern of the image in the x -direction.
- c. Determine the minimum sampling requirement needed to avoid aliasing in the x -

direction.

- d. Explain what will happen if the image is sampled below the required sampling rate.
- e. Suggest a suitable 8-bit quantization scheme for storing the image.
- f. Discuss the effect of quantization on image quality, storage size, and intensity accuracy.

Scenario

The image has a continuous sinusoidal intensity variation in the horizontal x direction and remains constant in the y direction.

a. Conversion of continuous image into digital image

The conversion occurs in two main stages:

1. Sampling

Sampling converts the continuous spatial coordinates into discrete coordinates.

For example:

$$x = 0, 1, 2, \dots, 255$$

and similarly for y.

The continuous image becomes a grid of pixels.

2. Quantization

Quantization converts continuous intensity values into a finite number of intensity levels.

For an 8-bit image:

$$2^8 = 256$$

possible intensity levels are available:

$$0 \text{ to } 255$$

Therefore:

Continuous Image $\rightarrow$ Sampling $\rightarrow$ Quantization $\rightarrow$ Digital Image

b. Spatial variation pattern

Since the intensity varies sinusoidally along x , the pattern can be represented as:

$$I(x) = A + B\sin(2\pi f_x x + \phi)$$

where:

- A = average intensity
- B = amplitude
- f_x = spatial frequency
- ϕ = phase

Since the intensity does not change along y:

$$I(x, y) = I(x)$$

Thus, the image contains **horizontal intensity variation**.

c. Minimum sampling requirement

According to the **Nyquist sampling theorem**, the sampling frequency must be at least twice the highest frequency present in the image.

$$f_s \geq 2f_{max}$$

Therefore:

$$\boxed{f_s \geq 2f_x}$$

This is the minimum sampling requirement to avoid aliasing.

d. Sampling below the required rate

If:

$$f_s < 2f_x$$

aliasing occurs.

The sampled image may show:

- False patterns
- Incorrect spatial frequencies

- Distorted sinusoidal patterns
- Loss of image information

Therefore, the original continuous pattern cannot be reconstructed correctly.

e. Suitable 8-bit quantization scheme

An **8-bit uniform quantization scheme** can be used.

The intensity range is divided into:

256

levels:

0,1,2, ...,255

If the intensity range is [0,255], each intensity can directly be represented by an 8-bit value.

f. Effect of quantization

Factor	Effect
Image quality	Small intensity differences may be lost
Storage	Each pixel requires 8 bits
Intensity accuracy	256 discrete intensity levels are available

Factor	Effect
Quantization error	Continuous values are approximated by discrete values

For a 256×256 image:

$$256 \times 256 \times 8 = 524,288 \text{ bits}$$

$$= 65,536 \text{ bytes}$$

$$\boxed{64 \text{ KB}}$$

Therefore, 8-bit quantization provides a good balance between image quality and storage requirements.

Question 3: Edge Detection in Thin Line Images

Scenario: A 512×512 grayscale image contains very thin vertical and horizontal white

lines of 1-pixel thickness on a dark uniform background. Such images are commonly

used in document analysis, engineering drawing interpretation, circuit diagram

processing, and road-lane detection.

The objective is to detect the line boundaries accurately without losing thin edge

information.

Answer the following:

a. Compare Sobel, Prewitt, and Canny edge detectors for detecting 1-pixel-thick

vertical and horizontal lines.

b. For a horizontal edge located near pixel position (256, 256), compute the Sobel

gradient response using the standard Sobel operator.

c. Explain how the gradient magnitude and gradient direction help in identifying the

strength and orientation of an edge.

d. Discuss the role of Gaussian smoothing in the Canny edge detection process.

e. Explain how Gaussian smoothing may affect edge localization, especially for thin

lines.

f. If the Canny threshold is reduced to half of its original value, explain the expected

changes in detected edges, weak edges, and false edges.

Scenario

A 512×512 grayscale image contains 1-pixel-thick horizontal and vertical white lines on a dark background.

a. Comparison of Sobel, Prewitt, and Canny

Feature	Sobel	Prewitt	Canny
Edge detection	Good	Good	Excellent
Noise handling	Moderate	Moderate	High
Edge localization	Moderate	Moderate	Very good
Thin edges	Good	Good	Excellent
Computational cost	Low	Low	Higher
Real-time suitability	High	High	Moderate/High
Gradient calculation	Yes	Yes	Yes

Sobel: Uses weighted gradient kernels and is effective for horizontal and vertical edges.

Prewitt: Uses simpler gradient masks and is computationally inexpensive.

Canny: Uses smoothing, gradient calculation, non-maximum suppression, and double thresholding. It generally provides better and thinner edge detection.

For 1-pixel lines, **Canny is generally preferred when accurate edge localization is important**, although excessive smoothing can cause very thin lines to weaken or disappear.

b. Sobel gradient response

The standard Sobel operators are:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$
$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

For a horizontal edge, intensity changes mainly in the **vertical direction**, so G_y gives the main response.

Assume a dark-to-white horizontal transition:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 255 & 255 & 255 \end{bmatrix}$$

Then:

$$G_y = (-1)(0) + (-2)(0) + (-1)(0) + (1)(255) + (2)(255) + (1)(255)$$
$$G_y = 1020$$

Therefore:

$$\boxed{G_y = 1020}$$

and approximately:

$$G_x = 0$$

Gradient magnitude:

$$G = \sqrt{G_x^2 + G_y^2}$$

$$G = \sqrt{0^2 + 1020^2}$$

$$\boxed{G = 1020}$$

The exact sign changes depending on whether the transition is dark-to-white or white-to-dark.

c. Gradient magnitude and direction

Gradient magnitude represents **edge strength**:

$$G = \sqrt{G_x^2 + G_y^2}$$

- Large magnitude $\rightarrow$ strong edge
- Small magnitude $\rightarrow$ weak edge

Gradient direction is:

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

It indicates the direction of maximum intensity change.

For a horizontal edge, the gradient is mainly vertical.

For a vertical edge, the gradient is mainly horizontal

d. Role of Gaussian smoothing in Canny

Gaussian smoothing is the first major stage of Canny edge detection.

It is used to:

- Reduce image noise
- Remove small unwanted intensity variations
- Prevent noise from being detected as edges
- Make gradient detection more stable

The image is convolved with a Gaussian kernel before calculating gradients.

e. Effect on thin-line localization

Gaussian smoothing can slightly spread or blur a very thin 1-pixel line.

As a result:

- Edge location can become less precise.
- Edge intensity can become weaker.

- Very thin lines may partially disappear if excessive smoothing is applied.

Therefore, a **small Gaussian kernel/sigma** should be used when detecting very thin lines.

f. Reducing Canny threshold by half

If the Canny threshold is reduced to half:

- More weak edges will be detected.
- Some previously undetected edges will appear.
- Detection sensitivity increases.
- False edges caused by noise may also increase.

Question 4: Feature Extraction for Handwritten Digit Recognition

Scenario: A computer vision system is being developed for recognizing handwritten

digits from scanned images. The handwritten digits vary in size, orientation, stroke

thickness, and writing style. Therefore, robust feature extraction is required before

classification.

Answer the following:

- a. Explain the working principles of SIFT, SURF, and ORB feature extraction methods.
- b. Compare SIFT, SURF, and ORB in terms of scale invariance, rotation invariance, computational cost, and real-time suitability.
- c. If a digit image is rotated by 45 degrees, identify which feature extraction methods can still produce reliable matching features.
- d. If the average number of keypoints detected per image is 120 and 50 images are processed, compute the total number of keypoints detected.
- e. Suggest a suitable dimensionality reduction technique to reduce the feature size while retaining 95% of the useful information.
- f. Explain how dimensionality reduction improves classification efficiency in handwritten digit recognition.

Scenario: Handwritten digits vary in size, orientation, stroke thickness, and writing style. Feature extraction is therefore required before classification.

a. Working principles of SIFT, SURF, and ORB

SIFT – Scale-Invariant Feature Transform

SIFT works through the following major stages:

1. Construct scale-space using Gaussian blurring.
2. Detect keypoints using Difference of Gaussian (DoG).
3. Assign orientation to keypoints.
4. Generate a descriptor based on local gradient patterns.
5. Match descriptors between images.

SIFT is robust to **scale and rotation changes**.

SURF – Speeded-Up Robust Features

SURF uses:

1. Integral images for fast computation.
2. Hessian-based detector for keypoint detection.
3. Approximation of Gaussian second-order derivatives.
4. Haar wavelets for orientation and descriptor generation.

SURF is designed to provide similar robustness to SIFT with improved computational speed.

ORB – Oriented FAST and Rotated BRIEF

ORB combines:

- **FAST** → keypoint detection

- **BRIEF** → feature description

ORB adds orientation handling to BRIEF, making the descriptor rotation-aware.

It is computationally efficient and suitable for real-time applications.

b. Comparison

Feature	SIFT	SURF	ORB
Scale invariance	Excellent	Excellent	Good
Rotation invariance	Excellent	Excellent	Good
Computational cost	High	Medium	Low
Descriptor	128-dimensional	64-dimensional commonly	Binary
Real-time suitability	Moderate	Good	Excellent
Robustness	Very high	High	Good

c. Rotation by 45°

A 45° rotation changes the orientation of image features.

SIFT, SURF, and ORB are designed to handle rotation changes.

SIFT and SURF are generally more robust, while ORB provides faster computation.

d. Total number of keypoints

Average keypoints per image:

120

Number of images:

50

Therefore:

$$120 \times 50 = 6000$$

6000 keypoints

e. Suitable dimensionality reduction technique

Principal Component Analysis (PCA) is a suitable technique.

PCA transforms the original features into a smaller number of principal components while retaining most of the information.

To retain 95% useful information:

PCA components that explain at least 95%

f. Benefits of dimensionality reduction

Dimensionality reduction:

- Reduces feature size.
- Reduces memory requirements.
- Reduces computational complexity.
- Makes classification faster.
- Removes redundant information.
- Can reduce overfitting.
- Makes machine learning models easier to train.

Therefore:

Feature Extraction → PCA → Reduced Features → Classifier → Digit Recognition

Question 5: Morphological Preprocessing for Character Recognition

Scenario: A binary image of size 256×256 contains scanned handwritten characters.

Due to poor scanning quality, the image contains small white noise dots in the

background and small black gaps inside the character strokes.

Before applying character recognition, the image must be cleaned using morphological

preprocessing techniques.

Answer the following:

a. Identify the morphological operation suitable for removing small white noise dots

from the background.

b. Identify the morphological operation suitable for filling small gaps or holes inside the

character strokes.

c. Explain the difference between opening and closing operations using simple image-

processing terminology.

d. If a 3×3 square structuring element is used, explain how it affects thin character

strokes.

e. Discuss the risk of using a large structuring element during preprocessing.

f. Propose a complete preprocessing pipeline for handwritten character recognition,

starting from image acquisition to feature extraction.

Scenario

A 256×256 binary image contains handwritten characters with small white noise dots and black gaps inside character strokes.

a. Removing small white noise dots

The suitable morphological operation is:

Opening

Opening consists of:

Erosion $\rightarrow$ Dilation

It removes small foreground objects/noise while preserving larger structures.

b. Filling small gaps or holes

The suitable operation is:

Closing

Closing consists of:

Dilation $\rightarrow$ Erosion

It fills small gaps, connects nearby components, and closes small holes.

c. Difference between opening and closing

Opening	Closing
Erosion followed by dilation	Dilation followed by erosion
Removes small foreground noise	Fills small gaps/holes
Separates weakly connected objects	Connects nearby objects
Can remove thin structures	Can fill small breaks

Simple representation:

Opening:

Erosion → Dilation

Closing:

Dilation → Erosion

d. Effect of a 3×3 structuring element

A 3×3 square structuring element examines a small neighborhood around every pixel.

It can:

- Remove very small noise.
- Fill small gaps.

- Smooth character boundaries.
- Connect nearby pixels.

However, if the character stroke is only **1 pixel thick**, erosion can remove or weaken that stroke.

Therefore, a 3×3 element should be used carefully for very thin handwriting.

e. Risk of a large structuring element

A large structuring element can cause excessive morphological modification.

Possible effects include:

- Thin strokes disappearing.
- Small character components being removed.
- Separate characters becoming connected.
- Character shapes becoming distorted.
- Important details being lost.

Therefore, the structuring element should be selected according to the size of noise and character strokes.

f. Complete preprocessing pipeline

A suitable handwritten character recognition pipeline is:

Image Acquisition



Grayscale Conversion



Noise Reduction



Thresholding / Binarization



Morphological Opening



Morphological Closing



Character Segmentation



Size / Orientation Normalization



Feature Extraction



Dimensionality Reduction (Optional)



Classification



Recognized Character

Explanation

1. Image Acquisition:

Capture or scan the handwritten document.

2. Grayscale Conversion:

Convert the color image into grayscale if required.

3. Noise Reduction:

Remove unwanted scanning noise using an appropriate filter.

4. Thresholding:

Convert the grayscale image into a binary image containing foreground and background.

5. Opening:

Remove small unwanted noise dots.

6. Closing:

Fill small gaps and reconnect broken character strokes.

7. Segmentation:

Separate individual characters from the document.

8. Normalization:

Resize and align characters to a standard size.

9. Feature Extraction:

Extract useful properties such as edges, contours, HOG features, or local features.

10. Dimensionality Reduction:

PCA can be applied if the feature vector is very large.

11. Classification:

Use a classifier such as SVM, k-NN, or a neural network.

12. Recognition:

The classifier predicts the handwritten character.